

Server-side para códigos de trial de federaciones

Comparativa de alternativas de hosting, arquitectura elegida, pseudocódigo de ambos lados y cómo se mantiene en productivo — para que federaciones como Unilivo o Livosam puedan autogestionar un período de RallyStats Premium gratis con un código, sin depender de un otorgamiento manual por cuenta.

Resumen: se recomienda **Firebase Cloud Functions (2nd gen) sobre plan Blaze**, reutilizando Firebase Auth y Firestore que ya están integrados en la app. A este volumen de uso (algunos cientos de canjes por federación) el costo real es \$0 — queda dentro de la franja gratis del plan Blaze — y es la opción que menos piezas nuevas suma al proyecto.

1. Comparativa de alternativas

Se evaluaron las tres formas de hospedar la lógica de canje (validar el código y otorgar el entitlement en RevenueCat) que son viables sin agregar infraestructura pesada:

- **Firebase Cloud Functions** (2nd gen, plan Blaze) — el mismo proyecto Firebase que ya usan Auth y Firestore.
- **Cloudflare Workers + Workers KV** — 100% gratis, pero es un proveedor nuevo.
- **Node/Express en un free tier tipo Render o Fly.io** — servidor tradicional, control total.

Criterio	Firebase Cloud Functions (Blaze)	Cloudflare Workers + KV	Node en Render / Fly (free)
Costo a este volumen	\$0 (franja gratis: 2M invocaciones/mes)	\$0 (100k req/día, sin tarjeta)	\$0 en la franja gratis del plan
Verificar quién llama (login)	Automático, con el Admin SDK	Manual: validar el JWT de Firebase a mano	Automático, con el Admin SDK
Dónde viven los códigos	Firestore, junto al resto de los datos de cuenta	Workers KV — un almacén nuevo y separado	Firestore, vía Admin SDK
Complejidad inicial	Baja	Media-alta (reimplementar verificación de auth)	Baja-media
Latencia / arranque en frío	Sin arranque en frío relevante	Sin arranque en frío (corre en el edge)	5-50s si estuvo inactivo
Proveedor nuevo para el proyecto	No — ya se usa Firebase	Sí	Sí
Día a día (deploy, logs, rollback)	firebase deploy; logs en la misma consola de Firebase	wrangler deploy; dashboard aparte de Cloudflare	git push; dashboard aparte del proveedor

Por qué gana Cloud Functions: a este volumen las tres opciones cuestan lo mismo (\$0), así que el criterio que decide es cuánta pieza nueva suma cada una. Cloud Functions no agrega proveedor ni sistema de identidad nuevo: usa el mismo uid de Firebase Auth que ya identifica a cada cuenta (`Purchases.logIn(uid)` en `SubscriptionController`) y guarda los códigos en el mismo Firestore que ya aloja `account_devices`. Cloudflare Workers sería igual de barato pero obliga a reimplementar a mano la verificación del token de Firebase (normalmente la resuelve el Admin SDK solo) y a separar los códigos del resto de los datos de cuenta. Render/Fly da control total pero el "sleep" por inactividad de los free tier agrega una demora la primera vez que alguien canjea después de un rato sin uso — no es grave para una acción puntual, pero no aporta nada sobre Cloud Functions a cambio de esa demora.

2. Cuándo implementarlo: plan en dos tiempos

Al momento de escribir esto, RallyStats 1.0.2 está enviada a producción y a la espera de la revisión de Google Play. El backend de canje no tiene por qué esperar a que esa revisión termine, pero tampoco conviene mezclarlo con ese envío — conviene separarlo en dos etapas.

Fase 1 — ahora, en paralelo a la revisión

El backend (Cloud Function + Firestore) es independiente del paquete que Google está revisando: se despliega con `firebase deploy` y no toca el APK/AAB para nada. Es un buen momento para adelantarlo justo mientras la publicación está bloqueada esperando revisión, sin que compita con nada urgente del lanzamiento.

- 1 Pasar el proyecto de Firebase a plan Blaze y configurar la alerta de presupuesto.
- 2 Crear la colección `federation_codes` y sus reglas de seguridad (cerradas a los clientes).
- 3 Escribir y desplegar `redeemFederationCode`.
- 4 Probarla de punta a punta contra el Firebase Emulator Suite y el modo sandbox de RevenueCat, sin tocar cuentas ni códigos reales.

Nada de esto requiere una nueva versión de la app ni interfiere en absoluto con la revisión en curso — son dos sistemas completamente separados.

Fase 2 — después de que la 1.0.2 esté estable en producción

El lado cliente sí necesita una versión nueva de la app (sumar la dependencia `cloud_functions`, la pantalla nueva, y pasar de nuevo por la revisión de Google) — por eso conviene separarlo del primer lanzamiento. Mezclar el debut en producción con código nuevo de canje dificulta diagnosticar cualquier problema que aparezca en los primeros días: ¿es por ser la primera vez en producción, o por el canje? Conviene esperar a que la 1.0.2 esté aprobada y estable (sin sobresaltos de crashes/ANR) antes de sumar esto.

- 1 Agregar `cloud_functions` al `pubspec.yaml` y el servicio `redeem_code_service.dart`.
- 2 Sumar la pantalla "Tengo un código" (`RedeemCodeScreen`) y su entrada en la navegación.
- 3 Generar el primer código real para Unilivo o Livosam y hacer una prueba piloto con una sola cuenta antes de repartirlo.
- 4 Seguir el proceso de release ya documentado del proyecto (regenerar el manual si corresponde, `flutter analyze`, APK, AAB, etc.) y subir todo como una versión nueva (ej. 1.0.3).

Mientras tanto: para cualquier federación que necesite el trial antes de que la Fase 2 esté lista, el otorgamiento manual por dashboard de RevenueCat (ver el documento aparte "Otorgar Premium gratis a una cuenta") sigue funcionando hoy mismo, sin depender de ninguna versión de la app.

3. Arquitectura elegida

El flujo agrega un único paso de red nuevo, puntual, al esquema que ya existe hoy (offline para los datos de juego; red sólo en login y en el refresh de suscripción):

- 1 La app (ya logueada) manda el código a una **Cloud Function callable** (`redeemFederationCode`).
- 2 La función valida el código contra una colección de **Firestore** (vigencia, cupo, uso previo de esa cuenta) dentro de una transacción.
- 3 Si es válido, la función llama a la **API de RevenueCat** (con la Secret API Key, que vive sólo en el servidor) para otorgar el entitlement `rallystats_pro` por la duración configurada.
- 4 La función responde éxito/error a la app; si fue éxito, la app llama a `SubscriptionController.refresh()` ahí mismo para que `isPremium` prenda sin esperar el próximo login.

Nada de esto reemplaza el otorgamiento manual por dashboard que ya está documentado aparte — sigue sirviendo para casos puntuales o de soporte. Este flujo es para cuando una federación necesita que muchas cuentas se autogestionen el trial sin que alguien lo haga a mano una por una.

4. Modelo de datos en Firestore

```
federation_codes/{code} // ej: "UNILIVO2026"
  federation: string // "Unilivo"
  durationBucket: string // "monthly" | "weekly" | ... (buckets de RevenueCat)
  maxRedemptions: number // cupo total del código
  redeemedCount: number // contador, se incrementa en la transacción
  expiresAt: timestamp // el código deja de servir después de esta fecha
  createdAt: timestamp
  createdBy: string // tu uid/mail, para auditoría

federation_codes/{code}/redemptions/{uid}
  redeemedAt: timestamp // 1 doc por cuenta que ya canjeó ESTE código
                          // (evita que la misma cuenta lo use dos veces)
```

Reglas de seguridad de Firestore para esta colección: cerrada por completo a los clientes (`allow read, write: if false;`) — sólo la Cloud Function, que corre con el Admin SDK, puede leerla y escribirla. Así un usuario no puede leer códigos ajenos ni inflar `redeemedCount` por su cuenta.

5. Pseudocódigo — lado RallyStats (Flutter)

Sigue el mismo patrón que ya usan `purchase_service.dart` / `device_addon_service.dart`: una función de servicio que devuelve un enum de resultado, y una pantalla simple que la llama.

lib/services/redeem_code_service.dart

```
enum RedeemOutcome {
  success,
  invalidCode,
  expired,
  quotaExceeded,
  alreadyUsedByAccount,
  networkError,
}

Future<RedeemOutcome> redeemFederationCode(BuildContext context, String code) async {
  final uid = AuthService.instance.currentUser?.uid;
  if (uid == null) return RedeemOutcome.networkError; // no debería pasar: hace falta login

  try {
    final callable = FirebaseFunctions.instance.httpsCallable('redeemFederationCode');
    final result = await callable.call({'code': code.trim().toUpperCase()});
    final data = Map<String, dynamic>.from(result.data as Map);

    if (data['ok'] == true) {
      // Prender isPremium ya mismo, sin esperar al próximo login/foreground.
      if (context.mounted) {
        await context.read<SubscriptionController>().refresh();
      }
      return RedeemOutcome.success;
    }
    return _mapErrorCode(data['error'] as String?);
  } on FirebaseFunctionsException catch (e) {
    // e.code: 'unauthenticated', 'unavailable', 'invalid-argument', etc.
    return RedeemOutcome.networkError;
  } catch (_) {
    return RedeemOutcome.networkError;
  }
}

RedeemOutcome _mapErrorCode(String? error) {
  switch (error) {
    case 'invalid_code': return RedeemOutcome.invalidCode;
    case 'expired': return RedeemOutcome.expired;
    case 'quota_exceeded': return RedeemOutcome.quotaExceeded;
    case 'already_used_by_account': return RedeemOutcome.alreadyUsedByAccount;
    default: return RedeemOutcome.networkError;
  }
}
```

lib/screens/subscription/redeem_code_screen.dart (esqueleto)

```
class RedeemCodeScreen extends StatefulWidget { ... }

// build():
// TextField para el código (autocapitalize, sin espacios)
// FilledButton "Activar" -> setState(loading = true)
// -> outcome = await redeemFederationCode(context, controller.text)
// -> setState(loading = false)
// -> switch(outcome): mostrar Snackbar/diálogo según el caso
// (success -> Navigator.pop con mensaje de éxito;
// el resto -> mensaje explicando el motivo, sin bloquear reintentar)
```

6. Pseudocódigo — lado backend (Cloud Function)

functions/src/redeemFederationCode.ts

```
export const redeemFederationCode = onCall(
  { region: 'southamerica-east1', secrets: [REVENUECAT_SECRET_KEY] },
  async (request) => {
    // 1) onCall ya verificó el ID token de Firebase; si no hay sesión, corta acá.
    if (!request.auth) throw new HttpsError('unauthenticated', 'Hay que iniciar sesion');
    const uid = request.auth.uid;
    const code = normalize(request.data.code); // trim + upper case

    // 2) Validar y reservar el cupo, todo en una transaccion de Firestore
    // (evita que dos canjes simultaneos exploten el cupo del codigo).
    const outcome = await db.runTransaction(async (tx) => {
      const codeRef = db.collection('federation_codes').doc(code);
      const codeSnap = await tx.get(codeRef);
      if (!codeSnap.exists) return { ok: false, error: 'invalid_code' };

      const c = codeSnap.data();
      if (c.expiresAt.toMillis() < Date.now()) return { ok: false, error: 'expired' };
      if (c.redeemedCount >= c.maxRedemptions) return { ok: false, error: 'quota_exceeded' };

      const redemptionRef = codeRef.collection('redemptions').doc(uid);
      if ((await tx.get(redemptionRef)).exists) {
        return { ok: false, error: 'already_used_by_account' };
      }

      tx.update(codeRef, { redeemedCount: FieldValue.increment(1) });
      tx.set(redemptionRef, { redeemedAt: FieldValue.serverTimestamp() });
      return { ok: true, durationBucket: c.durationBucket };
    });

    if (!outcome.ok) return outcome; // se lo pasa tal cual a la app

    // 3) Recien ahora, fuera de la transaccion, pegarle a RevenueCat.
    try {
      await grantPromotionalEntitlement(uid, outcome.durationBucket);
    } catch (err) {
      // Si RevenueCat falla, liberar el cupo reservado para no "quemar" el codigo en vano.
      await rollbackReservation(code, uid);
      throw new HttpsError('unavailable', 'No se pudo activar el premium, proba de nuevo');
    }

    return { ok: true };
  }
);

async function grantPromotionalEntitlement(uid: string, durationBucket: string) {
  const res = await fetch(
    `https://api.revenuecat.com/v1/subscribers/${uid}/entitlements/rallystats_pro/promotional`,
    {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${REVENUECAT_SECRET_KEY.value()}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ duration: durationBucket }),
    },
  );
  if (!res.ok) throw new Error(`RevenueCat respondio ${res.status}`);
}
```

El REVENUECAT_SECRET_KEY se guarda como secret de Cloud Functions (firebase functions:secrets:set REVENUECAT_SECRET_KEY), nunca en el repo ni mucho menos en la app — es la Secret API Key de RevenueCat, distinta de la Public API Key que ya usa purchases_flutter en el cliente.

7. Seguridad

- **Secret de RevenueCat** sólo del lado del servidor, como secret de Cloud Functions — nunca embebido en el APK.
- **Reglas de Firestore** cierran `federation_codes` a cualquier lectura/escritura directa del cliente; sólo la función (Admin SDK) accede.
- **Identidad** resuelta por `onCall` con el ID token de Firebase — no hay forma de canjear un código a nombre de otra cuenta.
- **Anti fuerza-bruta**: como los códigos son por federación (no por persona) y de pocos caracteres, conviene generarlos largos y poco adivinables (ej. 10-12 caracteres alfanuméricos) y, opcionalmente, sumar Firebase App Check para que la función sólo acepte llamadas que vengan de la app real.
- **Idempotencia**: el subdocumento `redemptions/{uid}` evita que la misma cuenta canjee el mismo código dos veces, incluso si reintenta tras un corte de red.

8. Cómo se mantiene en productivo

Despliegue

Las Cloud Functions se despliegan aparte del build de la app — cambiar la lógica del canje (por ejemplo, agregar un nuevo tipo de error, o ajustar la validación) no requiere subir un nuevo APK/AAB. Se agrega la carpeta `functions/` al repo, con su propio `package.json`, y se despliega con:

```
firebase deploy --only functions:redeemFederationCode
```

Conviene sumar este paso al proceso de release que ya está documentado en el proyecto (el que hoy cubre APK/AAB/Windows): si el cambio tocó algo en `functions/`, el deploy de funciones es un paso más, independiente de los builds de la app.

Entornos

Para no probar cambios contra los códigos reales de las federaciones, conviene un proyecto Firebase secundario de staging (o, más liviano al principio, el **Firestore Emulator Suite** corriendo Functions + Firestore en la propia máquina) combinado con el modo `sandbox` de RevenueCat para no otorgar entitlements de verdad mientras se prueba.

Logs y monitoreo

Cada invocación de la función queda en Cloud Logging automáticamente, visible desde la misma consola de Firebase (pestaña Functions → Logs). Conviene loguear explícitamente cada intento de canje (código, uid, resultado) para poder responder si una federación reclama que un código "no le funcionó". Una alerta de tasa de error en Cloud Monitoring avisa si la función empieza a fallar sistemáticamente (por ejemplo, si RevenueCat cambia algo en su API).

Costos y alertas

El plan Blaze requiere tarjeta cargada, pero no cobra nada mientras el uso quede dentro de la franja gratis (2 millones de invocaciones/mes) — a este volumen ese margen es enorme. De cualquier forma, conviene configurar una alerta de presupuesto en Google Cloud Billing (por ejemplo, avisar si se supera 1 USD en el mes) para enterarse de inmediato si algo empieza a consumir de más.

Rollback

Cada `firebase deploy` de una función 2nd gen crea una revisión nueva de Cloud Run por debajo. Si un deploy sale mal, se puede volver a la revisión anterior desde la consola de Cloud Run (o con `gcloud run services update-traffic`) sin tener que reescribir el código ni tocar la app.

9. Generación y administración de los códigos

No hace falta una pantalla de administración para esto al principio: alcanza con un script chiquito de Node (usando el Admin SDK, corrido a mano cuando una federación nueva pide el trial) que cree el documento en `federation_codes` con el cupo, la duración y el vencimiento que se acuerde con esa federación. Si en algún momento esto se vuelve frecuente, se puede envolver en una function HTTP protegida (sólo vos con un token de admin) o en una pantalla simple — pero eso ya sería una mejora sobre esta base, no un requisito para arrancar.

10. Relación con el otorgamiento manual por dashboard

Este flujo no reemplaza el otorgamiento manual desde el dashboard de RevenueCat (ver el documento aparte de "Otorgar Premium gratis a una cuenta") — son complementarios. El manual sigue siendo el camino más simple para un caso puntual o de soporte (una cuenta con un problema, una excepción). El código de canje sirve cuando una federación necesita que muchas cuentas se autogestionen el trial sin que alguien lo otorgue una por una.

11. Si el uso crece mucho más adelante

Incluso a diez veces el volumen actual (miles de canjes por mes, muchas federaciones a la vez), Cloud Functions seguiría dentro o muy cerca de la franja gratis del plan Blaze — el costo por invocación fuera de esa franja es fraccional. El cuello de botella antes llegaría del lado de RevenueCat (límites de su plan) que del lado de Firebase, así que no hace falta sobre-diseñar esto hoy pensando en escala.